Hindi Vidya Prachar Samiti's

RAMNIRANJAN JHUNJHUNWALA COLLEGE OF ARTS, SCIENCE AND COMMERCE (EMPOWERED AUTONOMOUS)

Soft Computing

Name: Surajkumar Yadav

Roll no: 10302

Class: MSc. Data Science and Artificial Intelligence Part I

Ramniranjan Jhunjhunwala College of Arts, Science and Commerce

Department of Data Science and Artificial Intelligence

CERTIFICATE

This is to certify Surajkumar Yadav of MSc. Data Science and Artificial Intelligence Roll No 10302 has successfully completed the practical of Soft Computing during the Academic Year 2024-2025.

Date:

(Prof. Sujata Kotian)
Prof-In-Charge

External Examiner

INDEX

Sr No.	Practical Name	Date	Sign
1	Implementation of single layer perceptron	31/01/25	
2	Implementation of multi-layer perceptron	28/01/25	
3	Perform backpropagation in neural network	28/01/25	
4	Implementation of fuzzy logic.	31/01/25	
5	Implementation of heb rule learning	21/01/25	
6	Implementation of self organizing map	01/02/25	
7	Implementation of delta rule learning	28/01/25	
8	Implementation of genetic algorithm.	07/02/25	

Practical 1: Implementation of single layer perceptron

[1]:
```python
import numpy as np

class Perceptron:
    def __init__(self, input_size, learning_rate=0.1, epochs=100):
        self.lr = learning_rate
        self.epochs = epochs
        self.weights = np.zeros(input_size + 1)  # +1 for bias

    def activation(self, x):
        return 1 if x >= 0 else 0  # Step function

    def predict(self, inputs):
        return self.activation(np.dot(inputs, self.weights[1:]) + self.weights[0])  # Weighted sum + bias

    def train(self, X, y):
        for _ in range(self.epochs):
            for inputs, label in zip(X, y):
                error = label - self.predict(inputs)
                self.weights[1:] += self.lr * error * inputs  # Update weights
                self.weights[0] += self.lr * error  # Update bias

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([0, 1, 1, 1])

# Train Perceptron
perceptron = Perceptron(input_size=2)
perceptron.train(X, y)

# Test Perceptron
for inputs in X:
    print(f"Input: {inputs}, Predicted Output: {perceptron.predict(inputs)}")
```

```
Input: [0 0], Predicted Output: 0
Input: [0 1], Predicted Output: 1
Input: [1 0], Predicted Output: 1
Input: [1 1], Predicted Output: 1
```

Practical 2: Implementation of multi-layer perceptron

[16]:
```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report
import matplotlib.pyplot as plt
```

[17]:
```
# Load the Breast Cancer dataset
data = load_breast_cancer()
X = data.data  # Features
y = data.target  # Binary target (0 = Benign, 1 = Malignant)
```

[18]:
```
# Split into train and test datasets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

[19]:
```
# Standardize the dataset
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

[20]:
```
# Define the MLP model
model = Sequential([
    Dense(64, activation='relu', input_shape=(X_train.shape[1],)),  # Hidden layer 1
    Dense(32, activation='relu'),                                   # Hidden layer 2
    Dense(1, activation='sigmoid')                                 # Output layer
])
```

[21]:
```
# Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Train the model
history = model.fit(X_train, y_train, epochs=50, batch_size=32, validation_split=0.2, verbose=1)

loss, accuracy = model.evaluate(X_test, y_test, verbose=1)
print(f"Test Accuracy: {accuracy:.2f}")

# Generate classification report
y_pred = (model.predict(X_test) > 0.5).astype("int32")
print(classification_report(y_test, y_pred))
```

```
Epoch 47/50
12/12 [==============================] - 0s 4ms/step - loss: 0.0058 - accuracy: 1.0000 - val_loss: 0.1085 - val_accuracy: 0.9560
Epoch 48/50
12/12 [==============================] - 0s 4ms/step - loss: 0.0056 - accuracy: 1.0000 - val_loss: 0.1095 - val_accuracy: 0.9560
Epoch 49/50
12/12 [==============================] - 0s 4ms/step - loss: 0.0052 - accuracy: 1.0000 - val_loss: 0.1110 - val_accuracy: 0.9560
Epoch 50/50
12/12 [==============================] - 0s 3ms/step - loss: 0.0049 - accuracy: 1.0000 - val_loss: 0.1102 - val_accuracy: 0.9560
4/4 [==============================] - 0s 1ms/step - loss: 0.0806 - accuracy: 0.9737
Test Accuracy: 0.97
4/4 [==============================] - 0s 1ms/step
              precision    recall  f1-score   support

           0       0.98      0.95      0.96        43
           1       0.97      0.99      0.98        71

    accuracy                           0.97       114
   macro avg       0.97      0.97      0.97       114
weighted avg       0.97      0.97      0.97       114
```

Practical 3: Perform backpropagation in neural network

[12]:
```python
import numpy as np
# Input (X) and Expected Output (Y)
X = np.array(([2, 9], [1, 5], [3, 6]), dtype=float)
Y = np.array(([92], [86], [89]), dtype=float)

# Normalize data
X = X / np.amax(X, axis=0)
Y = Y / 100
```

[13]:
```python
class NN(object):
    def __init__(self):
        self.inputsize = 2
        self.outputsize = 1
        self.hiddensize = 3
        # Initialize Weights
        self.W1 = np.random.randn(self.inputsize, self.hiddensize)
        self.W2 = np.random.randn(self.hiddensize, self.outputsize)

    def sigmoidal(self, s):
        return 1 / (1 + np.exp(-s))

    def sigmoidal_derivative(self, s):
        return s * (1 - s)

    def forward(self, X):
        self.z = np.dot(X, self.W1)  # Input to Hidden Layer
        self.z2 = self.sigmoidal(self.z)
        self.z3 = np.dot(self.z2, self.W2)  # Hidden to Output Layer
        self.op = self.sigmoidal(self.z3)
        return self.op

    def backward(self, X, Y, learning_rate=0.1):
        self.error = Y - self.op  # Compute error
        self.delta_output = self.error * self.sigmoidal_derivative(self.op)  # Output layer delta

        self.error_hidden = self.delta_output.dot(self.W2.T)  # Hidden layer error
        self.delta_hidden = self.error_hidden * self.sigmoidal_derivative(self.z2)  # Hidden layer delta

        self.W2 += self.z2.T.dot(self.delta_output) * learning_rate
        self.W1 += X.T.dot(self.delta_hidden) * learning_rate

    def train(self, X, Y, epochs=10000, learning_rate=0.1):
        for epoch in range(epochs):
            self.forward(X)
            self.backward(X, Y, learning_rate)
            if epoch % 1000 == 0:
                loss = np.mean(np.square(Y - self.op))
                print(f"Epoch {epoch}: Loss = {loss}")
```

[14]:
```python
# Train Neural Network
obj = NN()
obj.train(X, Y)

# Final Output
print("\nFinal Output after Training:")
print(obj.forward(X))
```

```
Epoch 0: Loss = 0.1128026945519326
Epoch 1000: Loss = 0.0005020751918747095
Epoch 2000: Loss = 0.0004287429753623694
Epoch 3000: Loss = 0.0003808298535231786
Epoch 4000: Loss = 0.0003409936635511415
Epoch 5000: Loss = 0.0003075910178512235
Epoch 6000: Loss = 0.00027937147083850403
Epoch 7000: Loss = 0.0002553590569944363
Epoch 8000: Loss = 0.0002347864960116792
Epoch 9000: Loss = 0.00021704608271367655

Final Output after Training:
[[0.90673138]
 [0.87934337]
 [0.88260157]]
```

Practical 4: Implementation of fuzzy logic.

[1]:
```
import numpy as np
import skfuzzy as fuzz
```

[2]:
```
# Step 1: Define the fuzzy variables (temperature and fan speed)
temperature = np.arange(0, 41, 1)  # temperature range (0 to 40 degrees)
fan_speed = np.arange(0, 11, 1)    # fan speed range (0 to 10)
```

[3]:
```
# Step 2: Define fuzzy membership functions for temperature
temp_low = fuzz.trimf(temperature, [0, 0, 20])     # Low temperature (0 to 20)
temp_moderate = fuzz.trimf(temperature, [10, 20, 30]) # Moderate temperature (10 to 30)
temp_high = fuzz.trimf(temperature, [20, 40, 40])   # High temperature (20 to 40)
```

[4]:
```
# Step 3: Define fuzzy membership functions for fan speed
fan_slow = fuzz.trimf(fan_speed, [0, 0, 5])      # Slow fan speed
fan_medium = fuzz.trimf(fan_speed, [0, 5, 10])    # Medium fan speed
fan_fast = fuzz.trimf(fan_speed, [5, 10, 10])     # Fast fan speed
```

[9]:
```
# Step 4: Fuzzify the input temperature value
temp_input = 30 # Example: Temperature input is 30 degrees

temp_low_degree = fuzz.interp_membership(temperature, temp_low, temp_input)
temp_moderate_degree = fuzz.interp_membership(temperature, temp_moderate, temp_input)
temp_high_degree = fuzz.interp_membership(temperature, temp_high, temp_input)
```

[10]:
```
# Step 5: Apply fuzzy rules
fan_slow_degree = min(temp_low_degree, 1)  # Rule 1: if temp is low, fan speed is slow
fan_medium_degree = min(temp_moderate_degree, 1)  # Rule 2: if temp is moderate, fan speed is medium
fan_fast_degree = min(temp_high_degree, 1)  # Rule 3: if temp is high, fan speed is fast
```

[7]:
```
# Step 6: Aggregate the results
aggregated_fan_speed = np.fmax(fan_slow_degree * fan_slow,
                               np.fmax(fan_medium_degree * fan_medium,
                                       fan_fast_degree * fan_fast))
```

[11]:
```
# Step 7: Defuzzify the result (crisp value)
fan_speed_output = fuzz.defuzz(fan_speed, aggregated_fan_speed, 'centroid')
print(f"Fuzzy Fan Speed Output for temperature {temp_input}°C: {fan_speed_output}")
```

```
Fuzzy Fan Speed Output for temperature 30°C: 8.333333333333334
```

Practical 5: Implementation of heb rule learning

[1]:
```
import numpy as np

x1 = np.array([1, 1, 1, -1, 1, -1, 1, 1, 1])
x2 = np.array([1, 1, 1, 1, -1, 1, 1, 1, 1])
b = 0
y = np.array([1, -1])
wtold = np.zeros((9,))
wtnew = np.zeros((9,))
wtnew = wtnew.astype(int)
wtold = wtold.astype(int)
```

[2]:
```
print("First input with target = 1")
for i in range(0, 9):
    wtnew[i] = wtold[i] + x1[i] * y[0]
wtold = wtnew
b = b + y[0]
print("New wt=", wtnew)
print("Bias value= ", b)
```

```
First input with target = 1
New wt= [ 1  1  1 -1  1 -1  1  1  1]
Bias value=  1
```

[3]:
```
print("second input with target = -1")
for i in range(0, 9):
    wtnew[i] = wtold[i] + x2[i] * y[1]
wtold = wtnew
b = b + y[1]
print("New wt=", wtnew)
print("Bias value= ", b)
```

```
second input with target = -1
New wt= [ 0  0  0 -2  2 -2  0  0  0]
Bias value=  0
```

Practical 6: Implementation of self organizing map

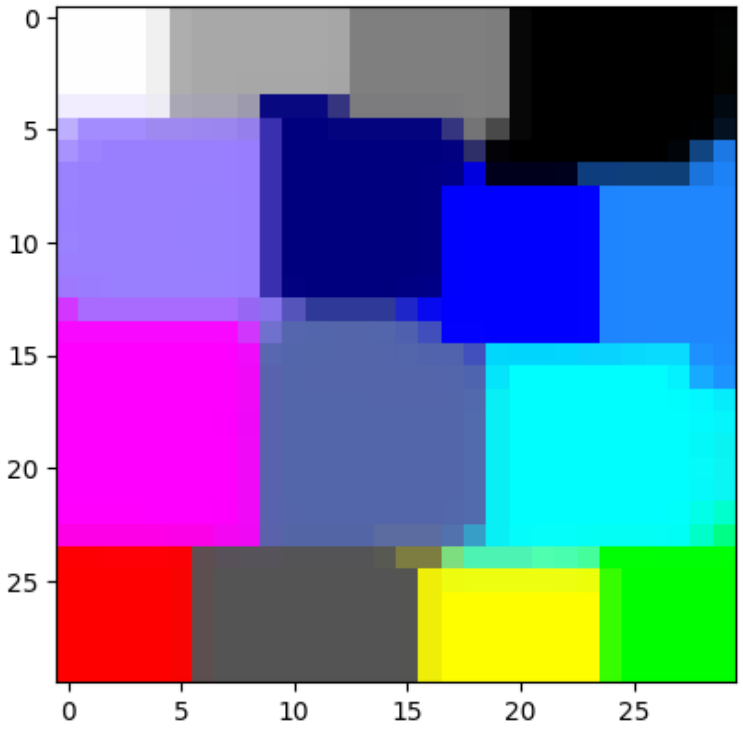
[11]:
```
from minisom import MiniSom
import numpy as np
import matplotlib.pyplot as plt
```

[12]:
```
colors = [
    [0.0, 0.0, 0.0], [0.0, 0.0, 1.0], [0.0, 0.0, 0.5],
    [0.125, 0.529, 1.0],[0.33, 0.4, 0.67],[0.6, 0.5, 1.0],
    [0.0, 1.0, 0.0],[1.0, 0.0, 0.0],[0.0, 1.0, 1.0],
    [1.0, 0.0, 1.0],[1.0, 1.0, 0.0],[1.0, 1.0, 1.0],
    [0.33, 0.33, 0.33],[0.5, 0.5, 0.5],[0.66, 0.66, 0.66],
]

color_names = ["black","blue","darkblue","skyblue",
    "greyblue","lilac","green","red",
    "cyan","violet","yellow","white",
    "darkgrey","mediumgrey","lightgrey",
]
```

[13]:
```
som = MiniSom(30, 30, 3, sigma=3.0, learning_rate=2.5, neighborhood_function="gaussian")
plt.imshow(abs(som.get_weights()), interpolation="none")
som = MiniSom(30, 30, 3, sigma=8.0, learning_rate=0.5, neighborhood_function="bubble")
som.train_random(colors, 500, verbose=True)
plt.imshow(abs(som.get_weights()), interpolation="none")
```

```
 [ 500 / 500 ] 100% - 0:00:00 left
 quantization error: 1.4023523065729547e-05
```



Practical 7: Implementation of delta rule learning

[1]:
```
import numpy as np

X= np.array([[1,2],[2,3],[3,4]])
t=np.array([0,1,0])
weights = np.random.rand(2)
eta = 0.1
```

[2]:
```
weights
```

[2]:
```
array([0.78032226, 0.91701747])
```

[3]:
```
def activation(x):
    return x

for i in range(100):
    total_error=0

    for i in range(len(X)):
        xi=X[i]
        target = t[i]
        y=activation(np.dot(weights ,xi))
        error = target -y
        total_error = total_error +error**2
        weights = weights +eta * error * xi

    if total_error < 0.01:
        break

print("Final weights : " ,weights)
```

```
Final weights :  [-3.39251939  2.10622796]
```

Practical 8: Implementation of genetic algorithm

[11]:
```python
import random

def genetic_algorithm(chromo_len, pop_size, generations, mutation_rate):
    population = [''.join(random.choice('01') for _ in range(chromo_len)) for _ in range(pop_size)]

    for gen in range(generations):
        fitnesses = [chromosome.count('1') for chromosome in population]
        best = max(zip(population, fitnesses), key=lambda x: x[1])
        print(f"Gen {gen + 1}: Best = {best[0]}, Fitness = {best[1]}")

        if best[1] == chromo_len: break  # Stop if optimal solution is found
        next_gen = []
        while len(next_gen) < pop_size:
            parents = random.choices(population, weights=fitnesses, k=2)
            point = random.randint(1, chromo_len - 1)
            child1 = parents[0][:point] + parents[1][point:]
            child2 = parents[1][:point] + parents[0][point:]
            next_gen.extend([
                ''.join(bit if random.random() >= mutation_rate else '1' if bit == '0' else '0' for bit in child1),
                ''.join(bit if random.random() >= mutation_rate else '1' if bit == '0' else '0' for bit in child2)
            ])
        population = next_gen[:pop_size]

    return best
```

[12]:
```python
# User input and execution
chromo_len = int(input("Binary string length: "))
pop_size = int(input("Population size: "))
generations = int(input("Number of generations: "))
mutation_rate = float(input("Mutation rate (0.0-1.0): "))
best = genetic_algorithm(chromo_len, pop_size, generations, mutation_rate)
print(f"\nOptimal Solution: {best[0]}, Fitness: {best[1]}")
```

```
Binary string length:  5
Population size:  20
Number of generations:  50
Mutation rate (0.0-1.0):  0.01
Gen 1: Best = 11011, Fitness = 4
Gen 2: Best = 01111, Fitness = 4
Gen 3: Best = 01111, Fitness = 4
Gen 4: Best = 11011, Fitness = 4
Gen 5: Best = 10111, Fitness = 4
Gen 6: Best = 11111, Fitness = 5

Optimal Solution: 11111, Fitness: 5
```